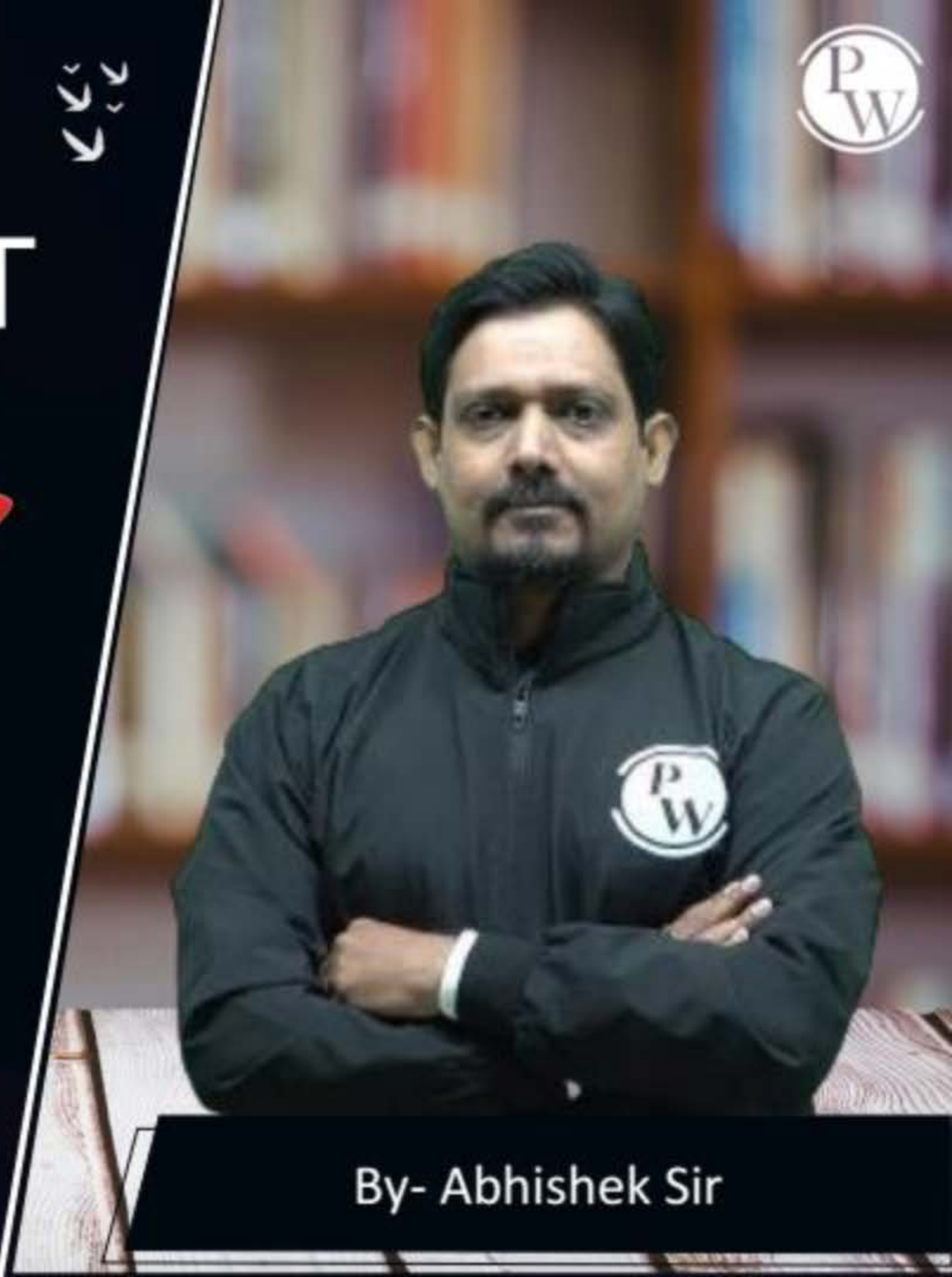
Computer Science & IT



C Programming

Array & Pointers

Lecture No. 05



By- Abhishek Sir

Recap of Previous Lecture



Topic Two Dimensional Array

Topic

Topic

Topic

Topic

Slide

Topics to be Covered



- Topic — Two Dimensional Array
- Topic — String
- Topic
- Topic
- Topic

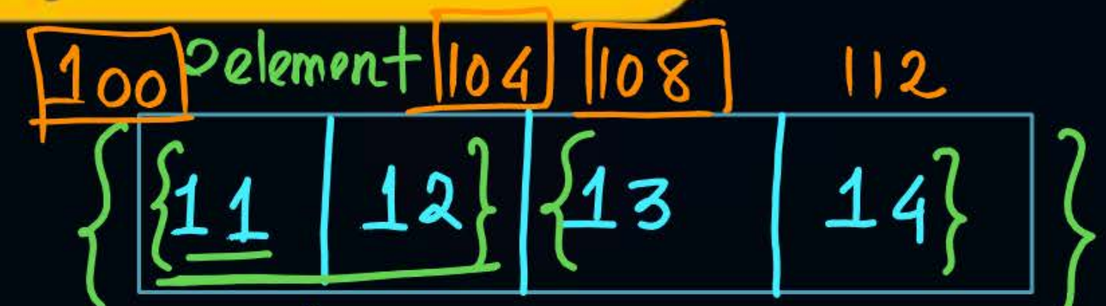
Slide

2-D Array



```
int a[2][2] =
{ {11,12},
  {13,14} };
```

2 D array
array of
1 D array



1 D array

a → 4B address of first element of 2-D array ✓

*a → address of first element 1-D integer array

**a → integer

&a → Address of 2D array

Slide



2-D Array

pointer Arithmetic

*ptr

ptr + 1 = ptr + 1 × Size of datatype

integer :-

integer address : 4B

int a[][] = {{ 11, 12}, { 13, 14}}

1.D Array address Datatype x No. of element 1D array = 4 x 2 = 8

2. D array address Datatype x No. of elements = 4B x 4 = 16B
(int, cha

Slide

Confidence

Life skill

2-D Array

```
#include <stdio.h>

int main(){
    int a[2][2] = {{11,12},{13,14}};

    printf("\n %u",a);
    printf("\n %u",&a);
    printf("\n %u",*a);
    printf("\n %u",**a);

    printf("\n %u",a+1);
    printf("\n %u",*a+1);
    printf("\n %u",**a+1);
    printf("\n %u",&a+1);

    return 0;
}
```

a
*a
**a

100 ← 1D array add
100 ← 2D array Add
100 ← integer Add
→ int

→ 100+1×8 = 108
→ 100+1×4 = 104
11+1 = 12
→ 100 + 1×4×4 = 116

2-D Array



Row →	Column 0	1	2	3
0	1 (100)	2 (104)	3 (108)	4 (112)
1	5 (116)	6 (120)	7 (124)	8 (128)
2	9 (132)	10 (136)	11 (140)	12 (144)

int a[3][4]

Size of 1D array = 4 × 4B = 16B (int size 4B)

Size of 2D array = 12 × 4B = 48B

$a[2][2] = *(*(a+2)+2)$

$*(*(100+2)+2)$

$*(*(100+2\times16)+2)$

$*(*(132)+2) = *(132+2) = *(140)$

↑ int add = 11

Slide

2-D Array

*a

Address of integer

a[1][2] = *(*(a+1)+2)

= *(*(100+1)+2)

= *(*(100+1×16)+2)

= *(*(116)+2)

= *(116+2) = *(116+2×4)

= *(124) = 7

a address of 1-D array

Slide



Declaration & initialization

Declaration

```
int a[][];
int a[][3];
int a[4][];
int a[4][3];
```

Initialization

```
int a[][] = {1,2,3,4,5,6};
int a[4][] = {11,12,13,14,15,16};
int a[][2] = {60,70,80,40};
int a[2][2] = {1,2,3,4};
int a[][] = {{1,2},{3,4}};   H.W
```

Slide



Question

```
#include <stdio.h>

int main(){
   int a[][3] = {11,12,13,14, 15, 16, 17 };

   for (int i=0;i<2;i++){
    for (int j=0;j<3;j++)
      printf("%d \t",a[i][j]);

    printf("\n");
   }
   return 0;
}
```



GATE 2015

What is the output of the following C code? (Assume that the address of X is 2000 (in decimal) and an integer requires four bytes of memory.)

```
int main() {
unsigned int x[4][3]={{1, 2,3},
                      {4,5, 6},
                     {7, 8, 9},
                     {10,11,12}};
 printf ("%u,%u,%u",x+3, *(x+3), *(x+2)+3);
      }
```

(A) 2036, 2036, 2036
(B) 2012, 4, 2204
(C) 2036, 10, 10
(D) 2012,4,6

X ← Address (1-D)
*X ← Addres (integer)
**X ← integer

address
address
address

Slide

GATE 2015

What is the output of the following C code? (Assume that the address of X is 2000 (in decimal) and an integer requires four bytes of memory.)

```
int main() {
unsigned int x[4][3]={{1, 2,3},
                      {4,5, 6},
                     {7, 8, 9},
                     {10,11,12}};
 printf ("%u,%u,%u",x+3, *(x+3), *(x+2)+3);
      }
```

(A) 2036, 2036, 2036
(B) 2012, 4, 2204
(C) 2036, 10, 10
(D) 2012,4,6

X - 1-D array address
*X - integer Address
** integer

X+3 = 2000+3×12 = 2000+36 = 2036

*(x+3) = *(2036) = 2036

Address integer

*(x+2)+3 = *(2000+2)+3

= *(2000+2×12)+3 = *(2024)+3 = 2024+3 = 2024+3×4 = 2036

int Add

Slide



Question

```
main () {/* Starting address 1002, Size of element is 2B */

int a[3][4]= { {1, 2, 3, 4},
               {5, 6, 7, 8},
               {9, 10, 11, 12}   };
printf ("\n%u %u %u ", a[0]+l, *(a[0]+l),*(*(a+0) + 1));
}
```

What is output of the above program?

(A) ✓ 1004 2 2
(B) 1002 2 2
(C) ✓ 1004 2 4
(D) ✓ 1004 4 4

1 1002	2 1004	3 1006	4 1008
5 1010	6 1012	7 1014	8 1016
9 1018	10	11	12
13	14	15	16

a[0]+1
= *(a+0)+1 → Add of 1D array
= *(1002+0)+1
= *(1002)+1
= 1002+1 = 1002+1×2
↳ Address of int
= 1004

Slide



Question

```
main () {/* Starting address 1002, Size of element is 2B */

int a[3][4]= { {1, 2, 3, 4},
               {5, 6, 7, 8},
               {9, 10, 11, 12}   };
printf ("\n%u %u %u ", a[0]+1, * (a[0]+1),*(*(a+0) + 1));
}
```

What is output of the above program?

(A) 1004 2 2
(B) 1002 2 2
(C) 1004 2 4
(D) 1004 4 4

address
integer
integer

$*(a[0]+1)$

$*(1004) = 2$

$*(*(a+0)+1)$

$= a[0][1]$

$= 2$

Slide



GATE 2020

Homework

Consider the following C program.

```
# include <stdio.h>
int main (){
int a[4][5] =  {{1, 2, 3, 4, 5},
                {6, 7, 8, 9, 10},
                {11, 12, 13, 14, 15},
                {16, 17, 18, 19, 20}};

printf("%d\n", *(*(a+**a+ 1)+4));
return (0);
}
```

The output of the program is _____.

Slide



String

String is array of characters : upon int

int a = 10; → integer

char ch = 'a'; → character

string

within "string" ← string

Slide

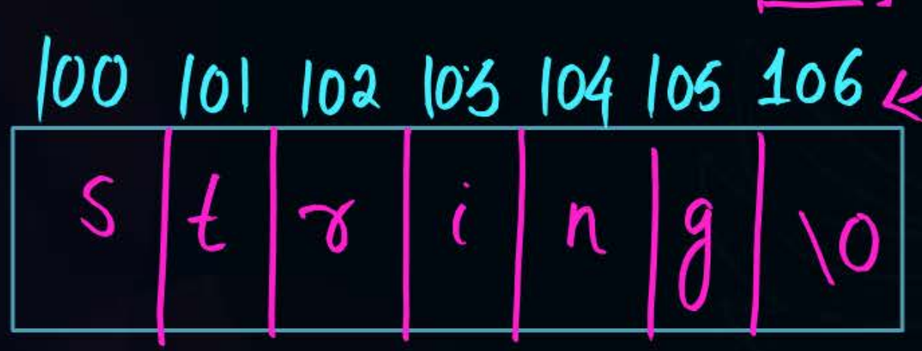


String

the string in c language is NULL terminated string

'\0' ← NULL character

ASCII value - 0



Slide



String

String is array of characters

datatype **char** ch[] = "string"; // Declaration and inilizahm using character array

Name of array represent address of fist element of array.

allowed 2 print value?

printf("%s", ch);
↳ address of fist character

printf("%s", ch+2); = ring

Slide

String



s	t	r	i	n	g	'\0'
100	101	102	103	104	105	106

Character Size = 1B

100+2

= 100+2×1

= 102

printf("%s", ch+6);

No output

Not allowed

ch = ch+1;

ch = "spring";

ch[1] = 'p'; allowed

printf("%s", ch);

Spring

Slide



String

```
Char ch[] = "string";
```

```
ch = ch + 1;   ← Not allowed.
```

```
ch = "spong";  ← Not allowed
```

DMA : malloc Linked list

Slide



2 mins Summary

- Topic
- Topic
- Topic
- Topic
- Topic

2D array

problem Solving

Strong

THANK - YOU

